

pa7uiclf

Ciência da Computação

Java Estrutura e Conceitos Sobre POO

05/09/2025

Java e POO

- **Introdução e características** do Java
- **Arquitetura detalhada** (JDK, JRE, JVM)
- **Estrutura básica** de programas Java
- **Sistema de tipos** (primitivos e referência)
- **Gerenciamento de memória** e Garbage Collection
- **Orientação a objetos** com exemplos práticos
- **Tratamento de exceções**
- **Collections Framework**
- **Recursos modernos** (Lambda, Streams)
- **Boas práticas** e convenções

Funcionamento da Linguagem Java

1. Introdução ao Java

Java é uma linguagem de programação orientada a objetos, desenvolvida pela Sun Microsystems (hoje Oracle) em 1995. Sua principal característica é ser "**Write Once, Run Anywhere**" (WORA), permitindo que código Java execute em diferentes plataformas sem modificações.

Características Principais:

- **Multiplataforma:** Roda em Windows, Linux, Mac OS, etc.
- **Orientada a Objetos:** Tudo é baseado em classes e objetos
- **Fortemente Tipada:** Tipos de dados devem ser declarados
- **Gerenciamento Automático de Memória:** Garbage Collector
- **Segura:** Controle rigoroso de acesso e execução
- **Robusta:** Tratamento de erros e exceções

2. Arquitetura e Funcionamento

2.1 Processo de Compilação e Execução

Código Java (.java) → Compilador (javac) → Bytecode (.class) → JVM → Código de Máquina

1. **Código Fonte (.java):** Arquivo de texto com código Java
2. **Compilação:** `javac` compila para bytecode (.class)
3. **Bytecode:** Código intermediário independente de plataforma
4. **JVM:** Interpreta/compila bytecode para código de máquina específico

Arquitetura Detalhada

2.2 Componentes da Plataforma Java

JDK (Java Development Kit)

- **Compilador (javac):** Converte .java em .class
- **JRE:** Ambiente de execução
- **Ferramentas:** debugger, javadoc, jar, etc.

JRE (Java Runtime Environment)

- **JVM:** Máquina virtual que executa bytecode
- **Bibliotecas:** Classes da API padrão do Java
- **Outros componentes:** necessários para execução

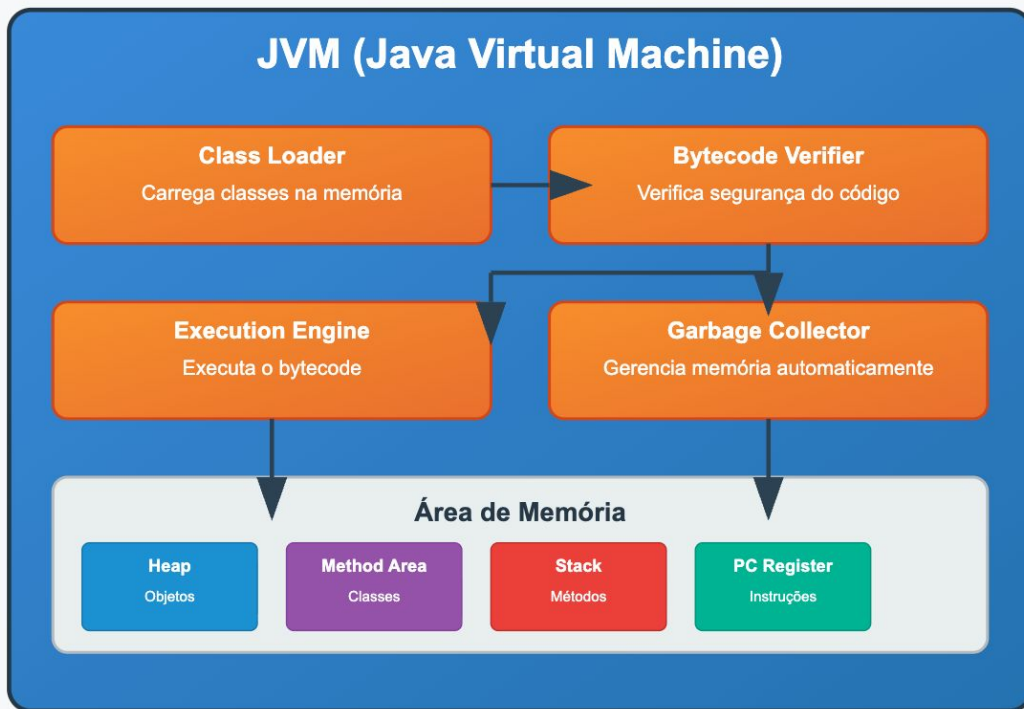
2.2 Componentes da Plataforma Java

JVM (Java Virtual Machine)

- **Class Loader:** Carrega classes na memória
- **Bytecode Verifier:** Verifica segurança do código
- **Execution Engine:** Executa o bytecode
- **Garbage Collector:** Gerencia memória automaticamente

Arquitetura Detalhada

Arquitetura da JVM



Estrutura Básica de um Programa Java

3. Estrutura Básica de um Programa Java

3.1 Anatomia de uma Classe Java

```
java

// Declaração do pacote
package com.exemplo.projeto;

// Importações
import java.util.Scanner;
import java.io.*;

// Declaração da classe
public class MinhaClasse {

    // Variáveis de instância (atributos)
    private String nome;
    private int idade;

    // Construtor
    public MinhaClasse(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    // Método principal (ponto de entrada)
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }

    // Métodos da classe
    public void exibirInformacoes() {
        System.out.println("Nome: " + nome + ", Idade: " + idade);
    }
}
```

Estrutura Básica de um Programa Java

3.2 Elementos Fundamentais

Pacotes (Packages)

- Organizam classes relacionadas
- Evitam conflitos de nomes
- Criam namespaces
- Exemplo: `java.lang`, `java.util`, `java.io`

Classes

- Moldes para criar objetos
- Contêm atributos e métodos
- Podem herdar de outras classes
- Podem implementar interfaces

Objetos

- Instâncias de classes
- Possuem estado (atributos) e comportamento (métodos)
- Criados com operador `new`

Sistema de Tipos

4. Sistema de Tipos

4.1 Tipos Primitivos

Tipo	Tamanho	Faixa	Exemplo
boolean	1 bit	true/false	boolean ativo = true;
byte	8 bits	-128 a 127	byte numero = 100;
short	16 bits	-32.768 a 32.767	short valor = 1000;
int	32 bits	-2^{31} a $2^{31}-1$	int idade = 25;
long	64 bits	-2^{63} a $2^{63}-1$	long populacao = 1000000L;
float	32 bits	IEEE 754	float altura = 1.75f;
double	64 bits	IEEE 754	double preco = 29.99;
char	16 bits	Unicode	char letra = 'A';

Sistema de Tipos

4. Sistema de Tipos

4.2 Tipos de Referência ou Wrapper

- **Classes:** String, Object, etc.
- **Arrays:** int[], String[], etc.
- **Interfaces:** List, Map, etc.
- **Enums:** Enumerações

Gerenciamento de Memória

5.1 Áreas de Memória da JVM

Stack (Pilha)

- Armazena variáveis locais e parâmetros de métodos
- Cada thread tem sua própria stack
- Liberada automaticamente quando método termina

Heap

- Armazena objetos e arrays
- Compartilhada entre todas as threads
- Gerenciada pelo Garbage Collector

Method Área

- Armazena informações das classes
- Constantes, código dos métodos
- Compartilhada entre threads

Gerenciamento de Memória

5.2 Garbage Collection

- **Automático:** Não é necessário liberar memória manualmente
- **Algoritmos:** Mark and Sweep, Generational, etc.
- **Quando ocorre:** Quando heap está cheia ou por solicitação
- **Otimização:** JVM escolhe melhor momento

Conceitos de Orientação a Objetos

6. Conceitos de Orientação a Objetos

6.1 Encapsulamento

- Oculta detalhes internos da implementação
- Controla acesso através de modificadores
- Métodos getter/setter para acesso controlado

java

```
public class ContaBancaria {  
    private double saldo; // Encapsulado  
  
    public double getSaldo() { // Acesso controlado  
        return saldo;  
    }  
  
    public void depositar(double valor) {  
        if (valor > 0) {  
            saldo += valor;  
        }  
    }  
}
```


Unimax
Centro Universitário
Max Planck

- Reutilização de código
- Relacionamento "é um"
- Palavra-chave **extends**

HERANÇA EM PROGRAMAÇÃO ORIENTADA A OBJETOS

```
graph TD; Object[Object  
(Classe raiz Java)] --> Veiculo[Veículo  
+ marca: String  
+ acelerar(): void  
+ frear(): void  
(Classe Base)]; Veiculo --> Carro[Carro  
+ numeroPortas: int  
+ acelerar(): void  
+ abrirPorta(): void  
estende Veículo  
(Overside acelerar)]; Veiculo --> Moto[Moto  
+ temCarenagem: boolean  
+ acelerar(): void  
+ empinar(): void  
estende Veículo  
(Overside acelerar)]; Veiculo --> Caminhao[Caminhão  
+ capacidadeCarga: double  
+ acelerar(): void  
+ carregarCarga(): void  
estende Veículo  
(Overside acelerar)]; Carro --> Sedan[Sedan  
+ temPortaMalas: boolean  
+ abrirPortaMalas(): void  
estende Carro]; Carro --> SUV[SUV  
+ tracacof4x4: boolean  
+ ativar4x4(): void  
estende Carro]; Caminhao --> Carreta[Carreta  
+ numeroEixos: int  
+ acoplarCarreta(): void  
estende Caminhao];
```

Exemplo em Java:

```
class Veiculo {  
    String marca;  
    void acelerar(){}  
}  
  
class Carro  
estende Veiculo {  
    // herda tudo!
```

LEGENDA

- Classe Raiz
- Classe Base
- Classes Filhas
- Herança

Características:

- Reutilização
- Especialização
- Polimorfismo
- Overside
- Relação "is-um"

Cada classe filha herda os atributos e métodos da classe pai, podendo adicionar novos ou modificar existentes

```
public class Veiculo {
    protected String marca;

    public void acelerar() {
        System.out.println("Acelerando...");
    }
}

public class Carro extends Veiculo {
    private int numeroPortas;

    @Override
    public void acelerar() {
        System.out.println("Carro acelerando...");
    }
}
```

Conceitos de Orientação a Objetos

6.3 Polimorfismo

- Mesmo método, comportamentos diferentes
- Override (sobrescrita) e Overload (sobrecarga)

6.4 Abstração

- Classes abstratas e interfaces
- Define contratos sem implementação
- Facilita manutenção e extensibilidade

java

```
public class Veiculo {
    protected String marca;

    public void acelerar() {
        System.out.println("Acelerando...");
    }
}

public class Carro extends Veiculo {
    private int numeroPortas;

    @Override
    public void acelerar() {
        System.out.println("Carro acelerando...");
    }
}
```

Conceitos de Orientação a Objetos

6.3 Polimorfismo

- Mesmo método, comportamentos diferentes
- Override (sobrescrita) e **Overload (sobrecarga)**

6.4 Abstração

- Classes abstratas e interfaces
- Define contratos sem implementação
- Facilita manutenção e extensibilidade

```
// SOBRECARGA DO MÉTODO acelerar
```

```
// 1. Acelerar sem parâmetros
```

```
public void acelerar() {  
    System.out.println("Veículo " + marca + " acelerando normalmente...");  
}
```

```
// 2. Acelerar com intensidade
```

```
public void acelerar(int intensidade) {  
    System.out.println("Veículo " + marca + " acelerando com intensidade " + intensidade);  
}
```

```
// 3. Acelerar com intensidade e duração
```

```
public void acelerar(int intensidade, int segundos) {  
    System.out.println("Veículo " + marca + " acelerando com intensidade " +  
        intensidade + " por " + segundos + " segundos");  
}
```

```
// 4. Acelerar com modo específico
```

```
public void acelerar(String modo) {  
    System.out.println("Veículo " + marca + " acelerando no modo: " + modo);  
}
```

```
}
```

Conceitos de Orientação a Objetos

O que é Abstração?

Abstração é o processo de **esconder detalhes complexos** e **expor apenas funcionalidades essenciais**. É como usar um controle remoto - você não precisa saber como funciona internamente, apenas apertar os botões!



Tipos de Abstração em Java:

1. Interface (100% abstrata)

- Todos os métodos são abstratos (exceto `default` e `static`)
- Não pode ter atributos de instância
- Uma classe pode implementar **múltiplas interfaces**
- Representa um "**contrato**" que as classes devem cumprir

2. Classe Abstrata (Abstração parcial)

- Pode ter métodos **abstratos** E **concretos**
- Pode ter atributos normais
- Uma classe só pode estender **uma classe abstrata**
- Representa uma "**base comum**" para classes relacionadas

Conceitos de Orientação a Objetos

Vantagens da Abstração:

1. **Simplificação:** Esconde complexidade desnecessária
2. **Reutilização:** Interface comum para diferentes implementações
3. **Manutenibilidade:** Mudanças internas não afetam quem usa
4. **Polimorfismo:** Permite tratar objetos diferentes de forma uniforme
5. **Contratos:** Força classes a implementar métodos essenciais

Exemplo: <https://trinket.io/java/790d57b71173>

Tratamento de Exceções

7. Tratamento de Exceções

7.1 Hierarquia de Exceções

Throwable

├─ Error (erros do sistema)

└─ Exception

├─ RuntimeException (não verificadas)

└─ Outras (verificadas)

Hierarquia de Exceções em Java



Legenda de Tipos

Throwable: Classe raiz **Error:** Erros do sistema **Exception:** Exceções tratáveis **RuntimeException:** Unchecked **Checked:** Tratamento obrigatório

Exemplos:
Implementações específicas

Pontos Importantes:

- **Throwable** é a superclasse de todos os erros e exceções
- **Error** representa problemas graves que normalmente não devem ser tratados
- **Exception** representa condições que uma aplicação pode querer capturar
- **RuntimeException** e suas subclasses são *unchecked*
- Todas as outras **Exception** são *checked*
- **Checked exceptions** devem ser tratadas ou declaradas
- **Unchecked exceptions** podem ser tratadas opcionalmente

Tratamento de Exceções

7. Tratamento de Exceções

7.2 Exceções checked e unchecked

Exceções Checked (Verificadas)

Características:

- Devem ser explicitamente tratadas pelo programador no momento da compilação
- O compilador força o tratamento através de `try-catch` ou propagação com `throws`
- Geralmente representam condições recuperáveis
- Herdam de `Exception` (mas não de `RuntimeException`)

Exceções Checked (Verificadas)

Exemplos comuns:

- `IOException` - problemas de entrada/saída
- `SQLException` - erros de banco de dados
- `ClassNotFoundException` - classe não encontrada
- `FileNotFoundException` - arquivo não encontrado

```
// Deve ser tratada obrigatoriamente
try {
    FileReader file = new FileReader("arquivo.txt");
} catch (FileNotFoundException e) {
    System.out.println("Arquivo não encontrado: " + e.getMessage());
}

// Ou propagada
public void lerArquivo() throws FileNotFoundException {
    FileReader file = new FileReader("arquivo.txt");
}
```

Tratamento de Exceções

7. Tratamento de Exceções

7.2 Exceções checked e unchecked

Exceções Unchecked (Não Verificadas)

Características:

- Não precisam ser explicitamente tratadas em tempo de compilação
- Geralmente indicam erros de programação
- Herdam de `RuntimeException`
- Podem ser tratadas opcionalmente

Exceções Checked (Verificadas)

Exemplos comuns:

- `NullPointerException` - referência nula
- `ArrayIndexOutOfBoundsException` - índice inválido
- `IllegalArgumentException` - argumento inválido
- `ArithmeticException` - erro aritmético

```
// Não obriga tratamento (mas pode tratar se quiser)
int[] numeros = {1, 2, 3};
try {
    System.out.println(numeros[5]); // Pode gerar exceção
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Índice inválido");
}

// Ou simplesmente deixar acontecer
int resultado = 10 / 0; // ArithmeticException em runtime
```

Tratamento de Exceções

7. Tratamento de Exceções

7.2 Exceções checked e unchecked

Boas Práticas

Para Checked Exceptions:

- Use quando a condição é recuperável
- Forneça informações úteis para recuperação
- Evite criar muitas checked exceptions desnecessárias

Para Unchecked Exceptions:

- Use para indicar erros de programação
- Documente quando métodos podem lançá-las
- Prefira validação preventiva quando possível

```
public class ContaBancaria {  
    private double saldo;  
  
    // Checked - condição recuperável  
    public void sacar(double valor) throws SaldoInsuficienteException {  
        if (valor > saldo) {  
            throw new SaldoInsuficienteException("Saldo insuficiente");  
        }  
        saldo -= valor;  
    }  
  
    // Unchecked - erro de programação  
    public void depositar(double valor) {  
        if (valor <= 0) {  
            throw new IllegalArgumentException("Valor deve ser positivo");  
        }  
        saldo += valor;  
    }  
}
```

Tratamento de Exceções

7. Tratamento de Exceções

7.2 Blocos Try-Catch-Finally

```
try {  
    // Código que pode gerar exceção  
    int resultado = 10 / 0;  
} catch (ArithmeticException e) {  
    // Tratamento específico  
    System.out.println("Divisão por zero!");  
} catch (Exception e) {  
    // Tratamento genérico  
    System.out.println("Erro: " + e.getMessage());  
} finally {  
    // Sempre executa  
    System.out.println("Limpeza de recursos");  
}
```

Collections Framework

8. Collections Framework

8.1 Interfaces Principais

- **List**: Sequência ordenada (ArrayList, LinkedList)
- **Set**: Sem duplicatas (HashSet, TreeSet)
- **Map**: Chave-valor (HashMap, TreeMap)
- **Queue**: Fila (PriorityQueue, LinkedList)

```
import java.util.*;

public class ExemploCollections {
    public static void main(String[] args) {
        // Lista
        List<String> nomes = new ArrayList<>();
        nomes.add("João");
        nomes.add("Maria");

        // Conjunto
        Set<Integer> numeros = new HashSet<>();
        numeros.add(1);
        numeros.add(2);

        // Mapa
        Map<String, Integer> idades = new HashMap<>();
        idades.put("João", 25);
        idades.put("Maria", 30);
    }
}
```

Collections Framework

9. Recursos Modernos (Java 8+)

9.1 Lambda Expressions

// Antes do Java 8

```
Comparator<String> comparador = new Comparator<String>() {  
    public int compare(String s1, String s2) {  
        return s1.compareTo(s2);  
    }  
};
```

// Com Lambda (Java 8+)

```
Comparator<String> comparadorLambda = (s1, s2) -> s1.compareTo(s2);
```

9.2 Streams API

```
List<String> nomes = Arrays.asList("João", "Maria", "Pedro", "Ana");
```

```
nomes.stream()  
    .filter(nome -> nome.length() > 3)  
    .map(String::toUpperCase)  
    .sorted()  
    .forEach(System.out::println);
```